real-mb-fidelity___fidelity-flat-f

An AgentMPI run analysis

Abstract

This is the manager-summarises-everything cell of the reduction-fidelity experiment: eight ranks, four uniquely identified facts each, all seven merges performed by rank 0 in sequence. No rank was erased. Retention is 1.000, all 32 identifiers reach the root, none is fabricated, and this is not merely the aggregate — I recovered all seven accumulators from this run's `spool/` directory under `runs/real-mb-fidelity/` and each carries exactly four identifiers per rank folded into it, 8, 12, 16, 20, 24, 28 and 32, growing by four at every application. The siblings `-binomial-f` (depth 3) and `-chain-f` (depth 7) score the same 1.000, so the tree-shape hypothesis finds no support in this campaign. It also could not have been tested here, and this cell shows why most clearly: the merges ran to 622 output tokens against a nominal 900-token budget, and the contract recorded in the fabric carries `max_tokens: null`, so the budget was a sentence in a prompt rather than a constraint. The first correction to make to the hypothesis as usually stated is about this run in particular: `flat` is not a single p-way fold at all. `algorithms.py` applies the *binary* operator $p-1$ times sequentially at the root, so this run has `fold_depth: 7` with `rounds: 1` — the trace shows seven successive `merge:d1`…`merge:d7` calls on rank 0 — and it is a depth-7 cell, not a depth-1 one. It is also the slowest of the three at 152.60 s against the tree's 58.25 s, despite needing the fewest communication rounds and sending the fewest tokens over the fabric, and it leaves seven of eight ranks completely idle for the entire run. The pattern is the worst of the three on latency and cost and has no compensating advantage on quality.

1 What this run is

property	value
run	`real-mb-fidelity__fidelity-flat-f`
experiment	–
campaign label	–
declared world size	8
ranks appearing in the log	8
executors	`broker` × 8, `worker` × 17
events	86
wall time	152.61 s
job outcome	completed

2 Headline measurements

metric	value	meaning
achieved parallelism	0.82×	total busy time over wall time
parallel efficiency	10.2%	achieved parallelism per rank
idle fraction	89.8%	rank-seconds paid for and unused
peak ranks busy	1	of 8 in the world
serial fraction of active time	100.0%	time with exactly one rank busy
load imbalance	8.00×	slowest rank's busy time over the mean
coordination share	12.5%	rank-seconds blocked in collectives, of those available
coordination in flight	100.0%	wall clock with any rank inside a collective
rank reattachments	17	fresh agent processes that took over a rank role
agent calls	7	invocations that reached a model
agent latency p50 / p95	20.51 / 26.56 s	per invocation
messages	7	7 eager, 0 rendezvous
tokens sent	987	0 deferred under rendezvous
tokens in / out	4,651 / 3,033	prompt and completion
cost	$0.0594	0.0196 $/kTok out

3 What the analysis notices

- [warning] Concurrency never exceeded one rank despite a world size of 8: this ran serially.
- [note] Load imbalance is 8.00×: the slowest rank was busy far longer than the mean.
- [ok] 17 rank reattachment(s) occurred, up to incarnation 8: agent processes were replaced mid-run while the rank roles, their mailboxes, and their context accounts persisted.
- [ok] All 1 checkable collective(s) sent exactly the number of messages their closed-form cost expression predicts.

4 Per-rank detail

rank	busy (s)	occ. (%)	tasks	calls	sent	recv	tok. sent	ctx (%)	state
0	124.71	81.7	7	7	0	7	0	0.0	finalized
1	0.00	0.0	0	0	1	0	141	0.0	finalized
2	0.00	0.0	0	0	1	0	141	0.0	finalized
3	0.00	0.0	0	0	1	0	141	0.0	finalized
4	0.00	0.0	0	0	1	0	141	0.0	finalized
5	0.00	0.0	0	0	1	0	141	0.0	finalized
6	0.00	0.0	0	0	1	0	141	0.0	finalized
7	0.00	0.0	0	0	1	0	141	0.0	finalized

Per-rank behaviour. Occupancy is busy time over the rank's own lifetime, so a rank that joined late is not penalised for the time before it existed.

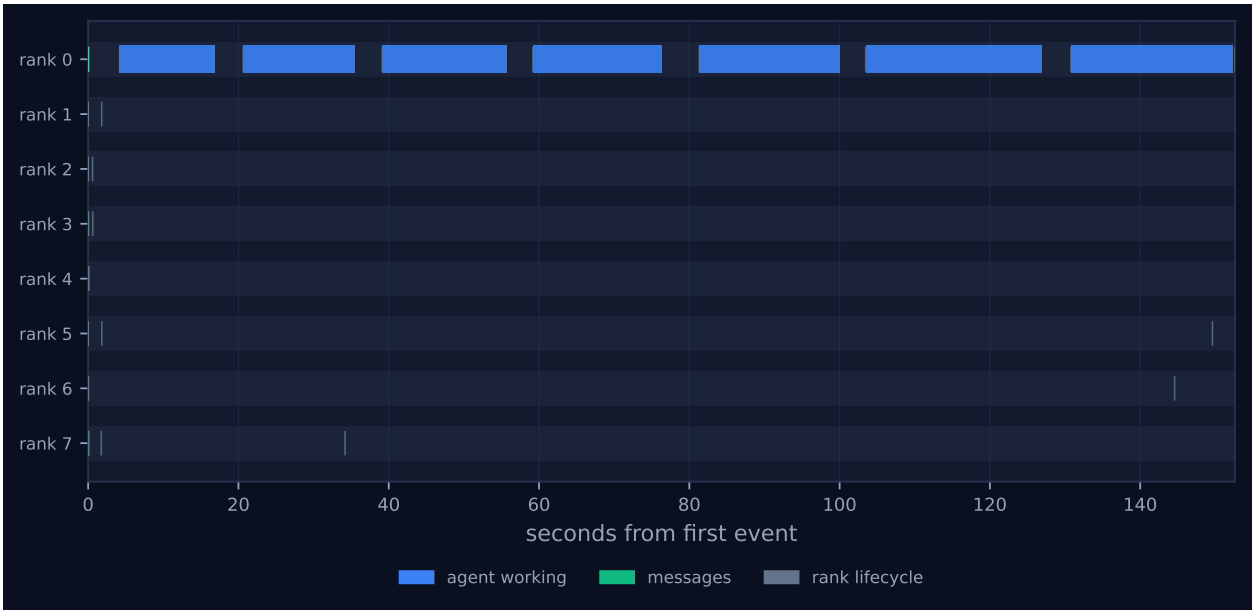

Figure 1: Execution timeline, one lane per rank. Bars are intervals when a rank held a task; ticks are messages; diamonds are window operations, lifecycle transitions, failures, and recovery actions. Drawn in the trace viewer's palette so the same shapes read the same way in both.

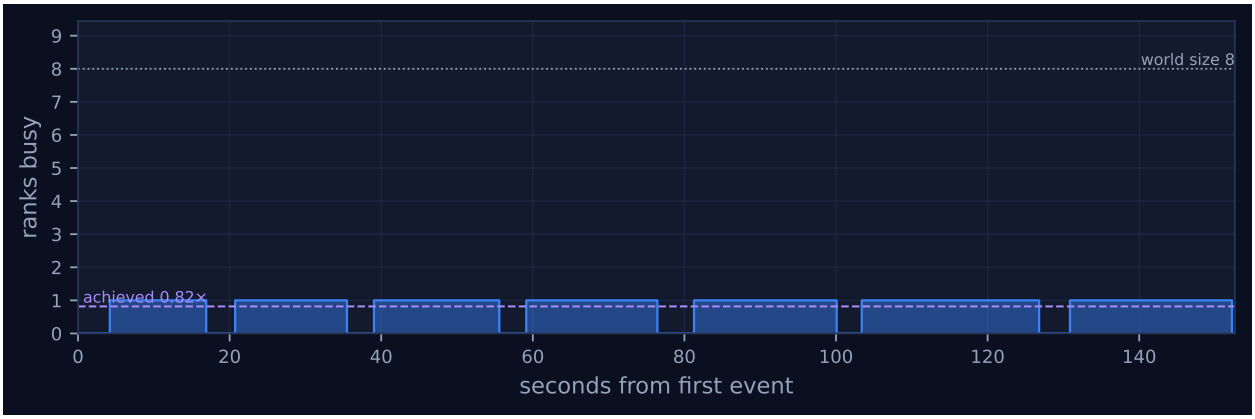

Figure 2: Ranks simultaneously busy over time. The dotted line is the declared world size and the dashed line the achieved parallelism; the gap between them is what the run paid for and did not use. Faint vertical lines mark collective invocations.

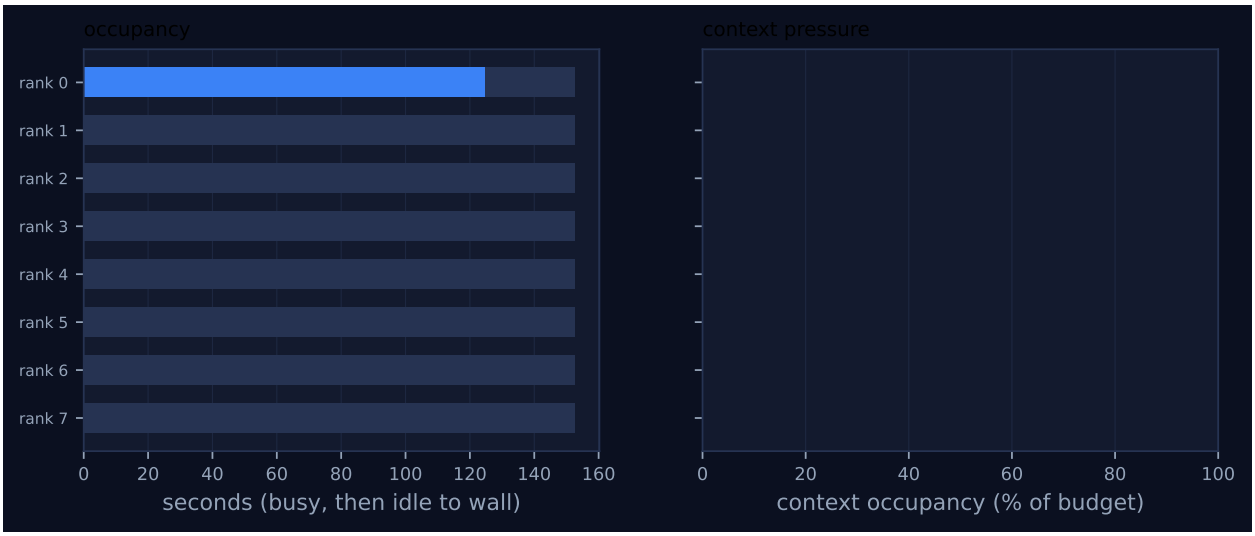

Figure 3: Per-rank occupancy and context pressure. Left: busy time, with the remainder to wall time in grey. Right: context occupancy against budget, amber above half and red above 80%, where admission pressure starts to reject artefacts.

5 Collectives, against the cost model

op	algorithm	label	p	seen	rounds	pred.	msgs	pred.	tokens	wall (s)	skew (s)	model
reduce	flat	–	8	8	1	1	7	7	987	152.60	152.60	✓

Messages are the count the fabric logged, attributed to each invocation through its internal tag, rather than the count the collective reported — the two are independently produced, and preferring the log is what makes this a check rather than a restatement. A dash in the model column means the comparison is not meaningful: either no closed form exists for that algorithm, or the invocation never completed.

6 Reading the timeline

Figure 1 shows one occupied lane and seven empty ones. Rank 0 carries a train of seven contiguous bars spanning the whole 152.6 s of the run; ranks 1 through 7 carry two or three instantaneous ticks each, all within the first 40 ms, and nothing else. That is the flat reduction drawn literally: every non-root rank sends its leaf report to rank 0 and returns, which all seven do by $t = 0.04$ s, and their `coll.reduce` records `fold_depth: 0` and a wall time of 1–3 ms. The root then does all the work. Peak ranks busy is 1 of 8, the serial fraction of active time is 100.0%, and rank 0's busy time of 124.71 s is the only nonzero entry in the per-rank table.

The bars visibly widen from left to right, and that is the run's most informative detail. The seven merges took 17.01, 18.51, 20.51, 20.51, 23.51, 27.01 and 25.51 s as the prompt grew from 442 tokens to 914 and the output from 189 tokens to 622. Seven times the median 20.51 s predicts 143.6 s against the measured 152.60 s, and the under-prediction is exactly this drift: unlike the chain, which spreads its growing accumulator over seven different ranks and whose latencies stay near their median, the flat root re-reads its own ever-larger accumulator every time. Pooling all 21 merges across the three cells, latency fits $12.31\,\text{s} + \text{output\_tokens}/57.0\,\text{tok/s}$ with $r = 0.789$; this run has

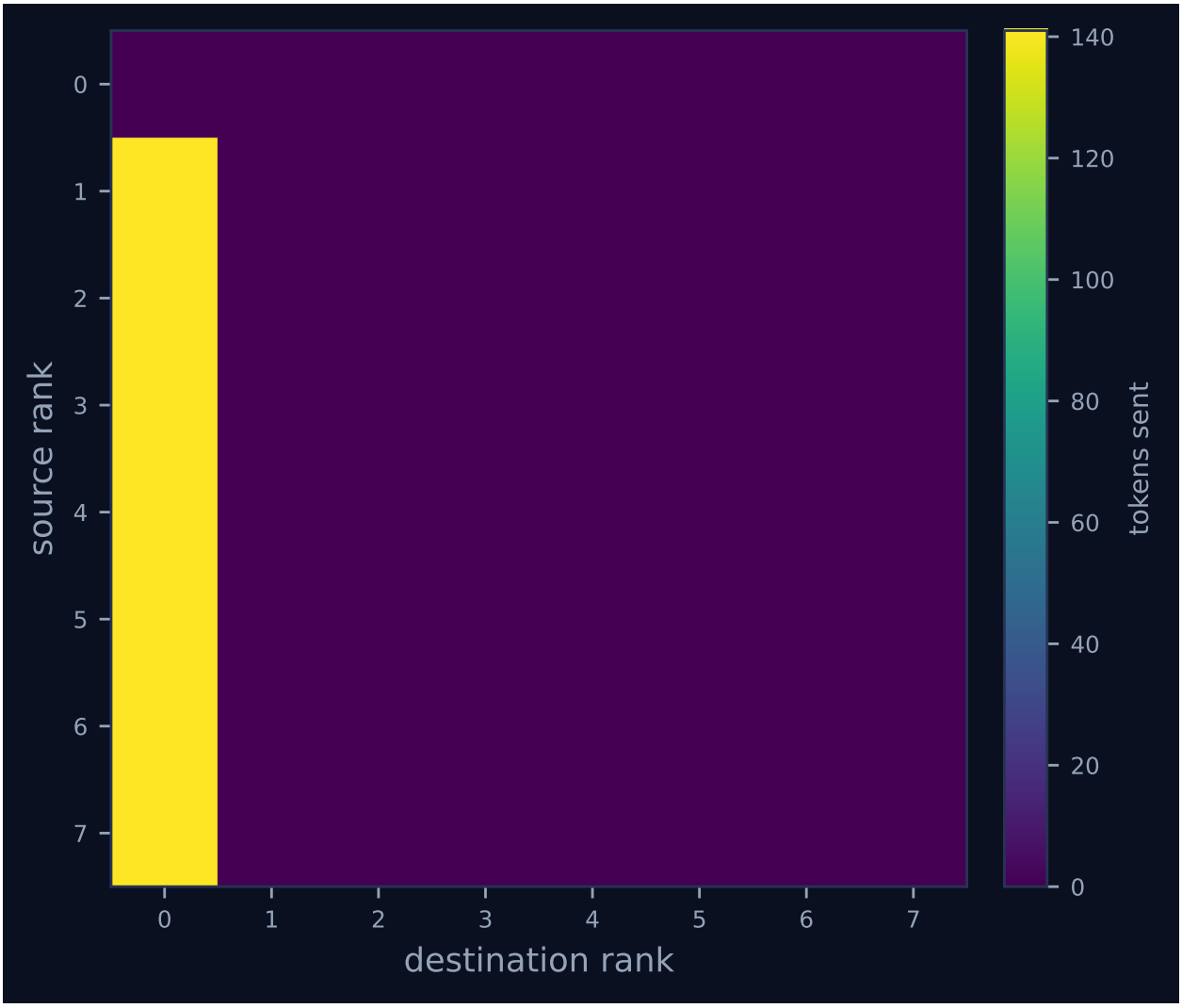

Figure 4: Communication matrix: tokens sent from each rank to each rank. The algorithm's shape is visible here — a tree concentrates traffic on a root, a ring forms a diagonal band, and an unintended all-to-all fills the plane.

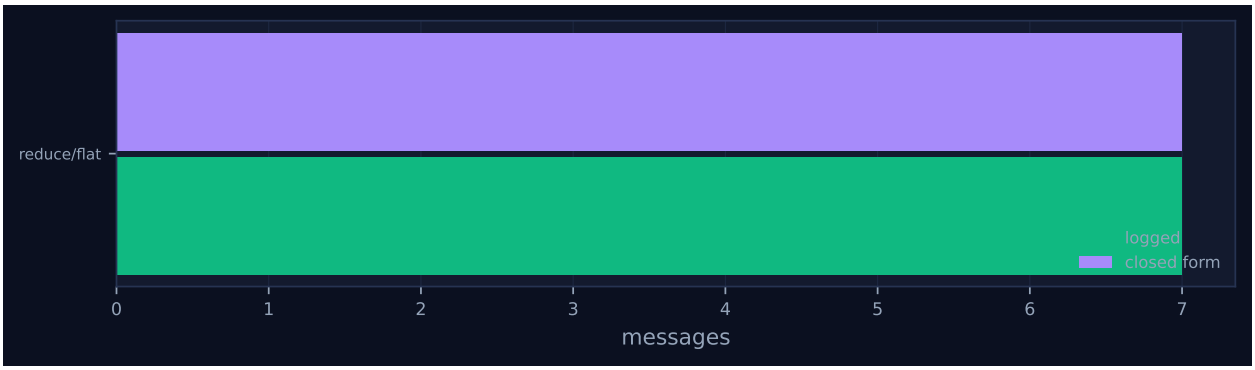

Figure 5: Logged message count against the closed-form prediction, per invocation. A red diamond marks a disagreement between the implementation and its own cost model.

both the largest single prompt in the campaign (914 tokens, which by itself exceeds the nominal 900-token output budget) and the highest latency percentiles (p50 20.51 s, p95 25.51 s, max 27.01 s, against the tree's 16.51 / 20.51 / 21.01).

The communication matrix, Figure 4, is a single column: seven edges into rank 0, all at exactly 141 tokens, totalling 987. That is the least fabric traffic of the three cells — against the tree's 1,483 and the chain's 2,638 — because only raw leaf reports ever travel, and no accumulator is ever sent anywhere. The 141 is 9–12 tokens less than the chain's and binomial's leaf messages because the `flat` branch sends the bare payload while the other two wrap it in an envelope carrying `depth` and `weight`. So the pattern that minimises inter-rank communication is also the one that maximises wall time and prompt tokens (4,651, the campaign's highest), which is the trade the cost model predicts and which this cell confirms.

There is one respect in which this run's trace is cleaner than its siblings': it has no post-completion tail. `job.finish` is emitted at 152.61 s and the log's last event is at 152.61 s, where the binomial cell's log runs 5.8× past its job completion and the chain's 2.1×. The dashboard's rank health table reflects the difference — rank 0 is shown `finalized` with no suspicion, while ranks 1–7 are shown `running` and suspected `fail_stop` in red, because their last worker incarnation attached and never finalised. That is a liveness heuristic misreporting a completed run: `metrics.json` records `ok: true`, `failed_ranks: []`, zero failures and `state: finalized` for all eight ranks. The same red rows appear on all three cells and are worth not misreading as a fault.

7 Interpretation

Most of the run's shape is true by construction. The world size, the four items per rank, the merge prompt, the 900-token nominal budget and the choice of the flat algorithm are harness decisions in `bench_fidelity`, and both checkable predictions hold — 7 messages against a closed form of 7 and 1 round against 1, so `model_checks_agree` is 1 of 1 and Figure 5 shows no red diamond. The construction that most needs naming is the one the hypothesis usually gets wrong. A flat reduce is often described as folding all p inputs at the root in one step, which would make it a depth-1 algorithm and the natural upper bound on retention. The implementation does no such thing: the `flat` branch of `algorithms.py` gathers all p contributions, sorts them by source rank because the operator is non-commutative, and then applies the binary operator $p - 1$ times in sequence, setting `fold_depth = p - 1 = 7` and `rounds = 1`. The trace confirms it directly: seven `agent.call` events at rank 0 labelled `merge:d1` through `merge:d7`, and seven accumulators in the spool growing $\{0,1\}$, $\{0,1,2\}$, …, $\{0,\ldots,7\}$ in rank order. Consequently this campaign contains only two distinct fold depths — 3 for the tree, 7 for this cell and the chain — rather than three, which halves the design's leverage on the variable under test. The genuine single-application-of-p-inputs form is the variadic kernel used only by `kary`; it is absent here and appears in the `sat-mb-fidelity` and `inc-mb-fidelity-inc` siblings at $k = 4$, depth 2.

The retention result is what emerged, and it gives the hypothesis no support. Retention is 1.000 in this run, in the depth-7 chain and in the depth-3 tree, with zero fabricated identifiers in all three. Because an aggregate figure can hide a reduction that keeps a complete prefix and erases the tail — and this cell is the one where that failure would be most natural, since the root folds in strict rank order and a budget-bound root would simply stop — I reconstructed the fold rather than trusting the aggregate. The seven accumulators carry 8, 12, 16, 20, 24, 28 and 32 identifiers over rank sets $\{0,1\}$ through $\{0,\ldots,7\}$: four per rank at every step, no thinning of the early prefix, no rank missing from the final result. Per-rank retention is genuinely 1.000 for all eight ranks.

Whole-rank erasure is a real and serious failure mode of semantic reduction, but it does not occur in this campaign; it occurs in `inc-mb-fidelity-inc`, where the payload is high-entropy serials and checksums that cannot be paraphrased shorter and the budget is enforced by contract, and there retention falls to 0.385 with ranks 0–2 retained completely, rank 3 keeping 1 of 12 items and ranks 4–7 keeping none. The decisive point for the hypothesis is that it is the *same* 0.385 with the same per-rank pattern and the same rank-position correlation of -0.86 at k-ary depth 2, binomial depth 3, and chain and flat depth 7. When every node's output is bound by the same budget B, the last application is the binding one, so the surviving item count is B over the per-item cost regardless of how the items arrived; losses have no slack to compound into. Depth costs nothing in quality under a uniform enforced budget, and collective selection is therefore a pure cost decision.

The reason nothing was lost in this configuration is arithmetic, and this run's own left fold measures it better than either sibling because it grows one rank at a time. Its accumulator sizes are 189, 274, 359, 444, 530 and 615 tokens for 2 through 7 ranks, which is $18.3 + 85.2 \times$ (ranks folded) to within one token — 85 tokens per rank, or 21 per item. Extrapolated to eight ranks that is 700 tokens; the 900-token budget is first reached at $p \approx 10.3$, about 41 items. Eight ranks contributing four short items each is below the saturation threshold, so the operator had slack. The observed eight-rank result is 622 tokens rather than 700, because the final merge re-encoded into a terser register, saving 78 tokens; at the resulting 19.4 tokens per item the budget's capacity is roughly 46 items. This is the cell that establishes the antecedent of the corrected claim — fold depth penalises fidelity only when the reduction is capacity-bound — rather than testing its consequent.

There is a second and stronger reason the retention number cannot bear the weight the hypothesis wanted to put on it: the budget was never enforced. The `agent_calls` row for every one of the seven merges records `contract` = `{"name": "MergedReport", ..., "max_tokens": null, "semantics": ""}` while `meta` records `{"max_tokens": 900}`. The budget went to the executor as an advisory generation hint, which a worker is free to ignore, and never to the contract checker that would have rejected an overrun and retried with the overrun quantified. All seven calls show `attempt: 1` and the run reports zero contract violations, which under a null bound is uninformative rather than reassuring. The `inc-mb-fidelity-inc` fabrics show the corrected regime for contrast: `max_tokens: 450` with non-empty semantics, and its `flat` cell contains eight calls rather than seven: `merge:d3` returned 472 tokens on attempt 1, was rejected for overrunning the 450-token bound, and was re-issued as attempt 2, which returned 437 — the enforcement path firing on precisely this algorithm. The cost of leaving it advisory is visible in `sat-mb-fidelity`, still advisory but with a 450-token budget and twelve items per rank: its merges emitted up to 698 tokens, 55% over the stated limit, and still scored retention 1.000. This run did not overrun only because 32 short items fit comfortably under 900. Both facts are needed to read it: the budget was not enforced, and it made no difference here because nothing came close to it.

What the run establishes is the cost of the manager-summarises-everything pattern, and it is the campaign's worst cell on every axis that matters. All three algorithms perform exactly seven operator applications and send exactly seven messages; this one puts all seven on a single rank's critical path and shows each of them the entire accumulated result so far. It finishes in 152.60 s against the tree's 58.25 s — 2.62× slower — and costs $0.0594 against $0.0439, 35% more, on 4,651 prompt tokens against 3,747. It is even slower than the depth-7 chain (130.57 s) despite the chain doing the same seven applications, and the reason is decomposable. Part is prompt growth: this run's per-call latencies climb from 17.01 s to 27.01 s as the root's prompt goes from 442 to 914 tokens, while the chain's fluctuate around their 18.51 s median with no trend (18.01, 14.01, 17.51, 20.51, 19.01, 22.51, 18.51 s) because each hop is a different rank reading a smaller share. The rest is worker attach cost, a number the metrics do not compute. Decomposing each call into

`broker.enqueue` → `broker.claim` → `broker.complete`, this run spent 26.22 s of its 150.93 s of aggregate call latency waiting for a worker to claim a pending call (17.4%), against 12.27 s of 118.67 s in the tree (10.3%) and 3.20 s of 128.79 s in the chain (2.5%). Rank 0 reached incarnation 8 for seven calls: a fresh worker process attached for each merge, at roughly 3.7 s apiece. A tree amortises that across concurrent branches and a chain hands each call to an already-idle rank; a serial fold at one rank pays it in full seven times. So flat's single communication round buys 987 tokens of saved fabric traffic and costs 94 s of wall clock.

All three flagged findings are correct, and the interesting thing about the list is what it omits. The warning that "concurrency never exceeded one rank despite a world size of 8" is expected by construction rather than a defect: the flat algorithm puts every application at the root, so `max_busy` of 1 and a serial fraction of 100.0% are what it must produce, and the warning is worth having precisely because a reader encountering this configuration in a real harness should be told. The 17 reattachments are the broker's normal mode, and the cost-model agreement is the check passing. What is not flagged, and should be, is a genuine defect in the derived metrics, and this cell is where it is most misleading. Load imbalance is reported as 1.00×, which the metric's own documentation glosses as perfect balance, in a run where seven of eight ranks did literally zero work. The cause is that `Analysis.imbalance` filters to ranks with `busy_s > 0` before taking the maximum over the mean, so a run with one busy rank is by definition perfectly balanced. Computed over all eight ranks the honest figure is $124.71/15.59 = 8.00\times$, the worst possible at $p = 8$. The related `coordination_share` of 12.5% understates the same thing for a different reason: the seven non-root ranks return from the reduce in milliseconds, so their 152 s of idleness never enters `collective_rank_seconds`, where the chain's blocked receives push the same quantity to 22.4%. This cell is more wasteful of the pool than the chain and both metrics say the reverse. By contrast, the achieved-parallelism figure of 0.82× is one of the few in the campaign that is not corrupted by the wall-time definition, because this run alone has no post-completion tail — the binomial cell's published 0.31× and the chain's 0.45× are both computed over spans that are mostly worker churn after `job.finish`, and corrected they read 1.83× and 0.96×, which restores the true ordering.

8 Threats to this reading

The most serious threat is that retention 1.000 may not measure information transport at all. In the compressible configuration `make_fact_report` builds each item as a throughput of $100 + 7 \times \text{rank} + i$ under workload `['nominal', 'degraded', 'saturated'][i % 3]`, so an item's payload is a deterministic function of its identifier, and `fact_retention` scores only whether the identifier appears. The threat is acute for this cell because its accumulator is a strictly increasing rank prefix in a visibly arithmetic sequence: after four ranks the root has seen throughputs 100–103, 107–110, 114–117 and 121–124, and the next rank's four values are trivially predictable. A root that had lost rank 6's message entirely could emit `F-6-0` through `F-6-3` with correct values and score as having retained them. Every claim here about what survived is a claim about identifier survival on a guessable payload; the `-inc` variant, with high-entropy serials scored by `value_retention` on the payload as well as the label, exists precisely because of this.

Second, the final result is byte-identical across all three algorithms, and I cannot fully exclude that the worker pool remembered rather than recomputed it. Digest `e1801060bfbe` (1,498 bytes) is the only blob shared by all three runs' content-addressed stores, and three different prompts — 914 tokens here, 838 in the chain, 832 in the tree — produced it. Two of this run's intermediates also coincide with the tree's, its $\{0, 1\}$ merge from an identical prompt and its $\{0, 1, 2, 3\}$ merge from a

different one. The runs shared one persistent pool of Cursor subagents and ran sequentially, and rank 0 performed the final merge in all three, so the mechanism for reuse existed. The evidence favours convergence, and the discriminating test is the wording register. The population uses a small set of discrete renderings, from the verbatim input at 119 bytes per item down to `[F-0-0] system-0.0: 100 units/s, nominal.` at 46, and a merge inherits its accumulator's register — this run is the clearest case, holding one 82-byte register unchanged across all six of its intermediate merges (84, 83, 82, 82, 81, 81 bytes per item) while the chain holds a different 67-byte register and the tree's four subtrees independently adopt four more. The only step in any of the three runs where the register is *not* inherited is the last, where 32 items must be rendered and all three paths land on the same minimal encoding; and the title is house style rather than coincidence, since all 21 merges open their title with `Consolidated report` and 20 of the 21 use the exact form `Consolidated report: component groups ...`, the sole exception substituting `from` for the colon. Decisively against wholesale reuse, this run and the chain share *only* the final result and no intermediate whatsoever — their fold orders are opposite and their accumulators disjoint — and this run's unique seven-rank accumulator, 2,330 bytes, exists in no other store, which a worker replaying remembered text would not have produced. Byte-identity of a free-form title across three runs remains a coincidence I can explain but not rule out.

Third, this is one trial of one campaign. The comparison against the tree is safe: per-call latency across all 21 merges spans 13.01–27.01 s with a standard deviation of 3.56 s, so the 94 s gap is many times the variability of a single call and is predicted by mechanism — identical application counts, seven on one rank's critical path instead of three levels of a tree. The 22 s gap between this run and the chain is not safe on one trial each. I have offered two mechanisms for it, prompt growth and worker attach cost, and both are measured here rather than assumed, but with a single trial per algorithm I cannot separate their combined 22 s from run-to-run variance in model output; treat the flat-versus-chain ordering as suggestive and the flat-versus-tree ordering as established. The campaign configuration records `reps: 5` and `trials: 32`, neither of which applies to the fidelity bench — those parameters belong to other benches, and reading them as replication here would be wrong.

Fourth, several accounting figures mean less than they appear to. Every `msg.recv` records `admitted: false` because the collective's internal receives pass `admit=False`, so context occupancy is 0% and high water 0 even on rank 0, which read 4,651 prompt tokens: this run exercises no context pressure and says nothing about admission, which is a real gap given that a flat reduction is precisely the pattern whose root should exhaust its context first. The `job.finish` payload reports `tokens_deferred: 987` while `metrics.json` reports 0, because the runtime's counter includes unadmitted receipts and the analysis's counts only rendezvous back-pressure. The reported `rank_position_correlation` of 0.0 is not a measurement of positional fairness but an undefined quantity: at retention 1.000 the per-rank variance is zero, the correlation's denominator vanishes, and the code returns 0.0 by a guard — which matters most for this cell, since a rank-ordered left fold is the shape where a positive correlation would be expected if any existed. And the $0.0594 is modelled, not billed: 3.0/Mtok in and 15.0/Mtok out from the calibration block reproduce it exactly from token counts estimated with `tokenizer_exact: false`.

Finally, the surrogate siblings must not be read as agent behaviour, and one of them is easy to misread in this run's favour. `fid-synth` and `fid-inc-smoke` use the `function` executor whose operator drops each item with fixed probability per application, so it *implements* the depth-loss model rather than testing it. It also does not sample it: the surrogate seeds its generator from the prompt *length*, so its loss is a deterministic function of prompt sizes rather than a draw, which is why `fid-inc-smoke` scores retention 1.000 at depth 7 with a nominal drop rate of 0.10. At $p = 16$

`fid-synth` reports retention 0.875 for flat at depth 15 against 0.703 for binomial at depth 4 — flat scoring best, the opposite of the model it implements — so not even the caricature produces a clean depth law here, and none of these rows is evidence about agent behaviour. The comparison this run belongs in is with the agent-executed cells only, and there the published "at equal quality" macro `\NFidFlat` is 198 s from `sat-mb-fidelity` rather than the 152.60 s measured here, because `make_tables.py` fills those macros from whichever non-capacity-bound broker block iterates last and labels neither.